Docker — Complete Notes

Containers · Build, ship, and run apps anywhere

A reader-friendly, all-in-one reference for Docker.

---

1. 🐋 What is Docker

Docker is a platform to **build, package, and run applications in containers** — lightweight, isolated units that bundle your app with **everything it needs** (code, runtime, libraries, config). A container runs the same on a laptop, a CI runner, or a production server.

> 💡 **Mental model:** a container is a **running instance of an image**. The image is the immutable recipe/snapshot; the container is the live process. *"Build once, run anywhere."*

Why it matters: consistent environments (kills "works on my machine"), fast startup, dense resource use, and a clean unit for CI/CD and microservices.

---

2. ⚖️ Containers vs Virtual Machines

	Container	Virtual Machine
Isolates	A process (shares the host **kernel**)	A whole machine (its own **OS kernel**)
Size	MBs	GBs
Startup	Milliseconds–seconds	Seconds–minutes
Overhead	Minimal	Hypervisor + full guest OS
Isolation	OS-level (namespaces + cgroups)	Hardware-level (stronger)
Density	Many per host	Fewer per host

> Containers **share the host kernel** — that's why they're small and fast, but isolation is weaker than a VM. Linux containers on Windows/macOS actually run inside a lightweight Linux VM.

---

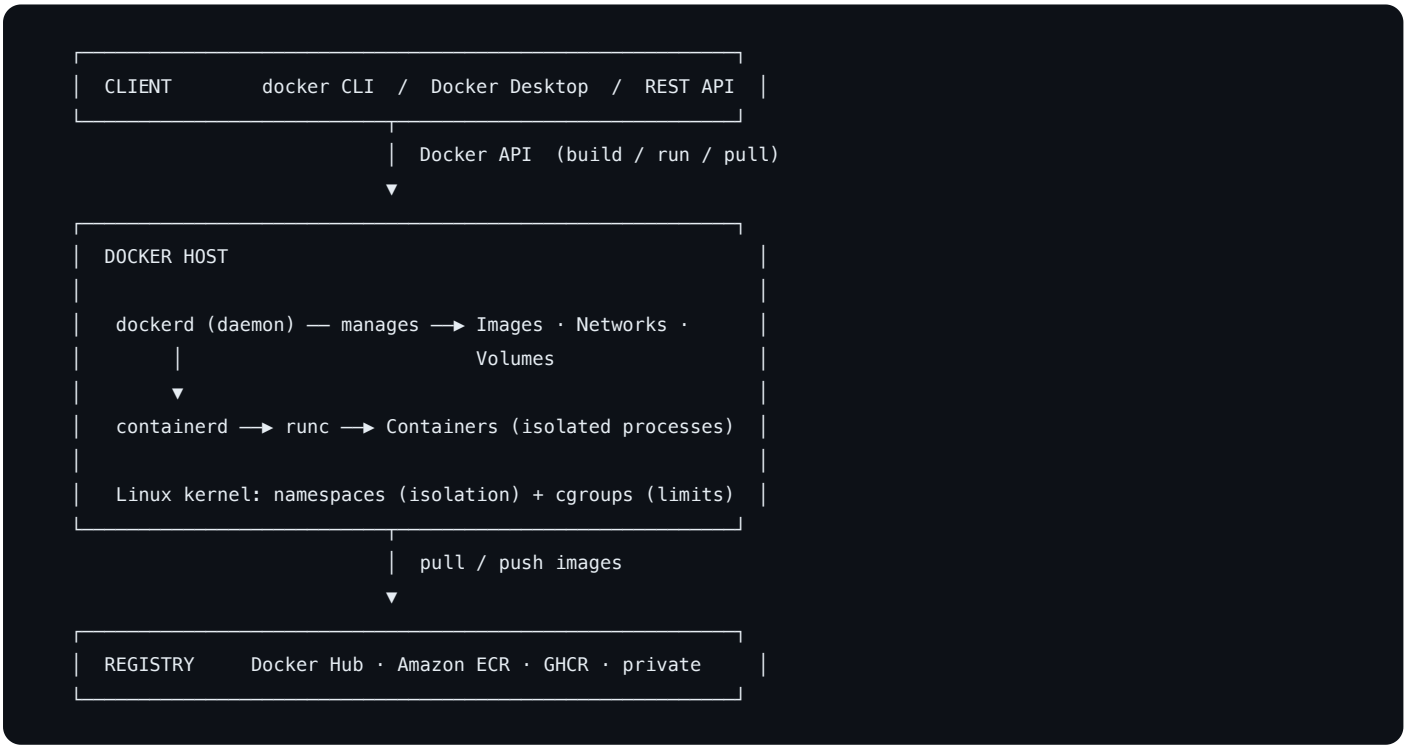
3. 🧩 Core Concepts

Concept	Description
Image	Read-only template (layers) used to create containers.
Container	A runnable instance of an image (an isolated process).
Dockerfile	Text recipe to build an image.
Registry	Store/distribute images (Docker Hub, ECR, GHCR…).
Volume	Persistent, Docker-managed storage.
Network	Virtual network connecting containers.
Docker daemon (`dockerd`)	Background service that builds/runs/manages objects.
Docker client (`docker`)	CLI that talks to the daemon (via REST API).
containerd / runc	Lower-level runtime that actually runs containers.

---

4. 🏗️ Docker Architecture

Docker uses a **client–server architecture** with three main parts: the **Client** (you), the **Docker Host** (the daemon plus everything it manages), and the **Registry** (where images live). The client talks to the daemon over a REST API; the daemon does the real work.



The components:

Component	Role
Docker Client (`docker`)	The CLI you type into. Sends commands (`build` , `run` , `pull` …) to the daemon over the Docker REST API (local socket or remote).
Docker Daemon (`dockerd`)	The brain/server. Builds images, manages containers, networks, and volumes, and talks to registries. Listens for API requests.
containerd	High-level container runtime the daemon delegates to — pulls images, manages the container lifecycle (start/stop), and storage.
runc	Low-level runtime that actually **spawns the container process** using kernel features (OCI-compliant).
Images	Read-only layered templates, cached locally on the host.
Containers	Running instances of images — isolated processes on the host kernel.
Registry	Remote image store (Docker Hub by default; or ECR, GHCR, private). The daemon pulls/pushes images here.

The kernel features that make it work:

Feature	Gives
Namespaces	**Isolation** — each container gets its own view of PIDs, network, mounts, hostname, users, IPC.
Control groups (cgroups)	**Resource limits** — caps CPU, memory, I/O per container.
Union filesystem (overlay2)	**Layered images** — stacks read-only layers + a writable layer (copy-on-write).

> 💡 The daemon — **not** the client — holds all state (images, containers, volumes). The client is just a thin front-end, which is why a local `docker` CLI can manage a remote Docker host.

---

5. ⚙️ How Docker Works (Lifecycle)

What actually happens when you run `docker run -p 8080:80 nginx` :

1. **Client → Daemon.** The CLI sends the run request to `dockerd` via the Docker API.
2. **Image resolve.** The daemon checks the **local cache** for `nginx` . If it's missing, it **pulls** the image layers from the **registry** (Docker Hub).
3. **Container create.** It assembles the container's filesystem by stacking the image's **read-only layers** and adding a thin **writable layer** on top (union FS, copy-on-write).
4. **Isolate & limit.** Via **containerd → runc**, the kernel creates **namespaces** (so the container has its own PID/network/mount view) and applies **cgroups** (CPU/memory limits).
5. **Networking.** The container is attached to a network (e.g. a `veth` pair into the `bridge`); `-p 8080:80` publishes the port to the host. Volumes/mounts are set up.
6. **Process start.** runc launches the image's **ENTRYPOINT/CMD** as **PID 1** inside the container. `STDOUT` / `STDERR` are captured as logs.
7. **Run & monitor.** The container runs as long as its main process lives. `docker ps` / `logs` / `stats` query the daemon for its state.

8. **Stop & remove.** `docker stop` signals the process to exit (SIGTERM → SIGKILL); the **writable layer persists** until `docker rm` removes the container.

Build flow (`docker build`): the daemon reads the **Dockerfile**, executes each instruction in a temporary container, and **commits the result as a new read-only layer** — reusing cached layers for unchanged steps — producing the final image.

> 🔑 Key insight: a container is **just a normal Linux process** on the host, wrapped in namespaces + cgroups and given a layered filesystem. There's no guest OS — that's why it starts in milliseconds.

---

6. 📦 Images & Layers

- An image is a **stack of read-only layers**; each Dockerfile instruction (`RUN` , `COPY` , `ADD`) adds a layer.
- Layers are **cached and shared** — unchanged layers are reused across builds and images (fast builds, small pulls).
- A running container adds a thin **writable layer** on top (copy-on-write); deleting the container discards it.
- Images are identified by **name:tag** (e.g. `nginx:1.27`) and a content **digest** (`sha256:…`).
- **Base image** = the starting layer (`FROM ...`), e.g. `alpine` , `ubuntu` , `python:3.12-slim` , or `scratch` (empty).

> 💡 Order Dockerfile steps from **least- to most-frequently changing** so the cache survives — put dependency installs before copying source code.

---

7. 📝 Dockerfile

A Dockerfile is the build recipe. Common instructions:

Instruction	Purpose
`FROM`	Base image to start from
`WORKDIR`	Set the working directory
`COPY` / `ADD`	Copy files in (`ADD` also untars / fetches URLs)
`RUN`	Execute a command at **build** time (creates a layer)
`ENV`	Set environment variables
`ARG`	Build-time variable
`EXPOSE`	Document the port the app listens on
`CMD`	Default command at **run** time (overridable)
`ENTRYPOINT`	The fixed executable (args from `CMD` /CLI)
`VOLUME`	Declare a mount point
`HEALTHCHECK`	How Docker tests container health
`USER`	Run as a non-root user

```
# Multi-stage build: small, no build tools in the final image
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM nginx:1.27-alpine
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
HEALTHCHECK CMD wget -qO- http://localhost/ || exit 1
CMD ["nginx", "-g", "daemon off;"]
```

> **CMD vs ENTRYPOINT:** `ENTRYPOINT` is the program; `CMD` supplies default args. Use `ENTRYPOINT ["app"]` + `CMD ["--flag"]` for a fixed binary with overridable arguments.

---

8. ✨ Image Best Practices

- **Small base images** — `alpine` / `-slim` / `distroless`; less to ship and a smaller attack surface.
- **Multi-stage builds** — compile in a "build" stage, copy only artifacts into a lean final stage.
- **`.dockerignore`** — keep `node_modules`, `.git`, secrets out of the build context.
- **Few, ordered layers** — combine related `RUN`s; put stable steps first for cache hits.
- **Pin versions** — `python:3.12.4-slim`, not `latest`, for reproducible builds.

- **Run as non-root** (`USER`), set a `HEALTHCHECK` , and avoid baking secrets into layers (they persist in history).
- **One main process per container** — scale by running more containers, not by stuffing many services into one.

---

9. 📦 Working with Containers

```
docker run -d --name web -p 8080:80 nginx:1.27   # detached, map host:container
docker run -it ubuntu bash                       # interactive shell
docker ps            # running containers (add -a for all)
docker logs -f web   # follow logs
docker exec -it web sh   # shell into a running container
docker stop web && docker start web
docker rm web        # remove (must be stopped, or -f)
```

- `-d` detached · `-it` interactive TTY · `-p host:container` port map · `-e KEY=val` env · `--rm` auto-remove on exit · `--restart unless-stopped` restart policy · `--memory` / `--cpus` resource limits.
- **Container states:** `created → running → paused → stopped/exited → removed` .

---

10. 💾 Data: Volumes & Bind Mounts

Containers are **ephemeral** — the writable layer dies with the container. Persist data outside it:

Type	What	Use
Volume	Docker-managed storage (`/var/lib/docker/volumes`)	**Preferred** for persistent data (DB files). Portable, backup-able.
Bind mount	Maps a host path into the container	Local dev (live-edit source), host config files.
tmpfs	In-memory only	Sensitive/temp data that must not hit disk.

```
docker volume create dbdata
docker run -v dbdata:/var/lib/postgresql/data postgres   # named volume
docker run -v "$(pwd)":/app node:20                       # bind mount (dev)
```

> Use **named volumes** for stateful services (databases) and **bind mounts** for live-reloading code during development.

---

11. 🌐 Networking

Docker creates virtual networks so containers can talk to each other and the outside.

Driver	Behavior
bridge (default)	Private network on the host; containers get internal IPs. On a **user-defined** bridge, containers resolve each other by **name** (built-in DNS).
host	Container shares the host's network stack (no isolation, no port mapping).
none	No networking.
overlay	Multi-host network (Swarm / clustered services).
macvlan	Container gets its own MAC/IP on the physical LAN.

- **Port publishing:** `-p 8080:80` exposes container port 80 as host 8080. Without `-p`, ports are reachable only within the Docker network.
- Create a network and attach containers so they reach each other by service name:

```
docker network create appnet
docker run -d --name db  --network appnet postgres
docker run -d --name api --network appnet -p 3000:3000 myapi   # api reaches db as "db"
```

---

12. 🗄️ Registries (Docker Hub & others)

- A **registry** stores and distributes images; a **repository** holds tagged versions of one image.
- **Docker Hub** is the default public registry. Others: **Amazon ECR**, **GitHub GHCR**, **GitLab**, **Google Artifact Registry**, or a self-hosted registry.

```
docker login                              # authenticate
docker pull nginx:1.27                     # download
docker tag myapp:1.0 user/myapp:1.0        # name for a repo
docker push user/myapp:1.0                 # upload
```

> Tag images meaningfully (semver + git SHA), and treat `latest` as "most recent build," **not** a stable pin for production.

---

13. 🧩 Docker Compose

Compose defines and runs **multi-container** apps from one YAML file — perfect for local dev and simple stacks.

```yaml
# compose.yaml
services:
  api:
    build: .
    ports: ["3000:3000"]
    environment:
      DATABASE_URL: postgres://app:secret@db:5432/app
    depends_on: [db]
  db:
    image: postgres:16
    environment:
      POSTGRES_USER: app
      POSTGRES_PASSWORD: secret
    volumes: ["dbdata:/var/lib/postgresql/data"]
volumes:
  dbdata:
```

```bash
docker compose up -d      # build + start the whole stack
docker compose ps         # status
docker compose logs -f    # tail logs
docker compose down       # stop & remove (add -v to drop volumes)
```

> Services on the same Compose project share a network and resolve each other by **service name** (`db` , `api`). Use one Compose file per app stack.

---

14. ⌨️ Essential CLI Commands

```bash
# Images
docker build -t myapp:1.0 .          # build from Dockerfile
docker images                        # list images
docker rmi myapp:1.0                 # remove image
docker history myapp:1.0             # inspect layers

# Containers
docker run / ps / logs / exec / stop / start / rm    # (see section 7)
docker inspect <id>                  # full JSON detail
docker stats                         # live resource usage
docker cp web:/etc/nginx.conf ./     # copy files in/out

# Housekeeping (reclaim disk)
docker system df                     # space usage
docker system prune -a               # remove unused images/containers/networks
docker volume prune                  # remove unused volumes
```

---

15. 🔐 Security Best Practices

- **Don't run as root** — add a `USER` ; consider rootless Docker.

- **Trusted, pinned base images** — official/verified, pinned by tag or digest; rebuild to get patches.
- **Scan images** for CVEs (`docker scout` , Trivy, Grype) in CI.
- **No secrets in images** — they persist in layer history; inject at runtime (env, secrets, mounted files).
- **Least privilege** — drop Linux capabilities (`--cap-drop ALL`), avoid `--privileged` , use read-only root FS (`--read-only`) where possible.
- **Resource limits** (`--memory` , `--cpus`) to contain noisy/abusive containers.
- **Keep the daemon secured** — the Docker socket is root-equivalent; never expose `dockerd` unprotected.

---

16. 🚀 Beyond Docker (Orchestration)

Docker runs containers on **one host**. Production at scale needs orchestration — scheduling, self-healing, scaling, rolling updates, service discovery across **many** hosts:

Tool	Role
Kubernetes (K8s)	The de-facto standard orchestrator.
Docker Swarm	Docker's built-in, simpler orchestrator.
Amazon ECS / EKS	AWS-managed container orchestration.
Nomad	HashiCorp's scheduler.

> Docker images are **OCI-compliant** — the same image you build runs on Kubernetes, ECS, etc. Docker for building/local dev; an orchestrator for running fleets in production.

---

17. 🧠 Quick Mental Model

- **Docker = package an app + its environment into a portable container.**
- **Image** = immutable layered recipe; **container** = a running instance of it.
- Build images with a **Dockerfile** (order steps for cache; use **multi-stage** + small bases).
- Containers are **ephemeral** — persist data with **volumes**, share code in dev with **bind mounts**.
- Containers talk over **networks** (resolve by name on user-defined bridges); expose with `-p` .
- Share via **registries** (`pull` / `tag` / `push`); pin versions, don't trust `latest` in prod.
- Run multi-container stacks with **Compose**; run fleets in prod with **Kubernetes/ECS**.
- Secure it: **non-root, pinned + scanned images, no baked-in secrets, least privilege.**

---

18. ❓ Common Interview Questions

Rapid-fire questions interviewers ask about Docker:

- **Q: Container vs VM?** — A container shares the host **kernel** (isolated process via namespaces + cgroups) → MBs, starts in ms. A VM runs a full guest OS on a hypervisor → GBs, slower, stronger isolation.

- **Q: Image vs container?** — Image = immutable layered template. Container = a running instance of an image (image + a thin writable layer).
- **Q: How do you make images small?** — Small base (alpine/slim/distroless), multi-stage builds, `.dockerignore`, few ordered layers, pin versions.
- **Q: CMD vs ENTRYPOINT?** — ENTRYPOINT = the fixed executable; CMD = default args (overridable). Combine for a fixed binary with overridable arguments.
- **Q: How do you persist data?** — Volumes (Docker-managed, preferred for state) or bind mounts (host path, good for dev). The writable layer is lost when the container is removed.
- **Q: How do containers communicate?** — On a user-defined bridge network they resolve each other by **name** (built-in DNS); publish ports to the host with `-p`.
- **Q: What provides isolation and limits?** — Linux **namespaces** (isolation) and **cgroups** (CPU/memory limits).
- **Q: Docker vs Kubernetes?** — Docker builds/runs containers on one host; Kubernetes orchestrates many containers across many hosts (scheduling, scaling, self-healing).

---

19. 🎯 Detailed Interview Q&A and When to Use

> Scenario-style interview questions with model answers and **when to use** guidance. Aimed at ~2 years of hands-on experience. The signal-handling thread (Q8 → Q15 → Q43 → Q44) is a deliberate trap chain interviewers love.

Fundamentals

1. What is Docker / the real problem it solves? Packages an app with all its dependencies into a portable image that runs identically anywhere with a container runtime. The real answer: it makes the runtime environment a versioned artifact — consistent dependencies, isolation, fast startup, density — not just "works on my machine." **When to use:** Containerize → microservices, consistent envs, CI/CD, scalable stateless apps. Don't → heavy stateful monoliths better on managed services, GUI/desktop apps, kernel-module/hardware-bound workloads, single-server scripts where it adds no value.

2. Image vs container; what's shared vs isolated? Image = immutable, read-only template (layers). Container = a running instance with a thin writable layer on top. Many containers share one image's read-only layers (copy-on-write); each gets its own isolated writable layer, process namespace, and network.

3. Container vs VM; be precise about the kernel. A VM virtualizes hardware and runs its own guest OS kernel via a hypervisor. A container shares the host kernel and is isolated via namespaces/cgroups — just a process. Lighter and faster, but less isolated. **When to use:** Container → density, fast startup, microservices, same-OS workloads. VM → strong isolation, different OS/kernel, untrusted multi-tenant.

4. Containers share the host kernel — security implication? A kernel exploit or container escape can reach the host and other containers — the boundary is the kernel, not a hypervisor. Run as non-root, drop capabilities, use seccomp/AppArmor. **When to use:** Plain containers → trusted internal workloads. gVisor / Kata / Firecracker → untrusted/multi-tenant code (running other people's code).

5. What is a Dockerfile; what does `docker build` do? A declarative recipe of instructions to assemble an image. `docker build` executes each instruction, creating a cached layer per instruction, and produces a final tagged image.

6. run vs start vs exec vs attach? `run` = create + start a new container. `start` = restart an existing stopped container. `exec` = run an additional command in a running container. `attach` = connect your terminal to PID 1's stdio. **When to use:** run → new container. start → relaunch a stopped one. exec → debug/shell into a running container. attach → watch/interact with PID 1.

7. COPY vs ADD; why prefer COPY? Both copy files. ADD also auto-extracts local tarballs and can fetch URLs. Prefer COPY because it's explicit; ADD's magic causes surprises and security risk. **When to use:** COPY → always. ADD → only to auto-extract a local tarball (rarely; never for URLs).

8. CMD vs ENTRYPOINT; both? exec vs shell form? ENTRYPOINT = the fixed executable; CMD = default args (or default command). With both, CMD's values are passed as args to ENTRYPOINT. Exec form (`["nginx","-g"]`) runs the binary directly as PID 1 — receives signals. Shell form (`nginx -g`) runs via `/bin/sh -c` , so the shell is PID 1 and your process doesn't get a clean SIGTERM. **When to use:** ENTRYPOINT → fixed binary the image always runs. CMD → default/overridable args. Exec form → always for the main process (signals). Shell form → only when you need shell features (pipes, env expansion) and don't care about signals.

9. What does EXPOSE actually do? (trap) Documentation/metadata only — it declares intent. It does not publish or open the port. You publish with `-p` at runtime (or `ports:` in Compose).

Images, Build & Optimization

10. Image layer; why does caching matter? Each instruction creates a content-addressed layer. On rebuild, Docker reuses cached layers until the first changed instruction, then rebuilds everything after. Good ordering = fast incremental builds.

11. Scenario — one code change re-runs `npm install` every build. You copied source before installing deps, so any file change busts the install layer's cache. Fix: copy only `package*.json` , run `npm install` , then `COPY . .` . Dependencies re-install only when the manifest changes.

12. RUN vs CMD vs ENTRYPOINT — when do they execute? RUN executes at build time (bakes results into a layer). CMD and ENTRYPOINT execute at runtime (define what the container runs).

13. Multi-stage build — conceptually, and why for security? One stage with the full toolchain builds, then `COPY --from=builder` only the final artifact into a minimal runtime base. The final image excludes compilers, build tools, source, and build-time secrets — shrinking the attack surface, not just size. **When to use:** Compiled languages (Go/Rust/Java) or any build with a heavy toolchain — ship only the artifact in a minimal image.

14. `.dockerignore` — why it both speeds builds and prevents leaks? It excludes files from the build context. Without it, `node_modules` , `.git` , `.env` , and secrets get copied in — bloating context (slow) and baking secrets/git history into the image (leak). Always ignore `.git` , `node_modules` , `.env` , build artifacts.

15. stop vs kill; signals; why care about SIGTERM? `stop` sends SIGTERM, waits a grace period (default 10s), then SIGKILL. `kill` sends SIGKILL immediately. Your app should trap SIGTERM to flush, close connections, and finish in-flight requests (graceful shutdown). **When to use:** stop → graceful shutdown (default). kill → force-terminate a hung/unresponsive container.

16. Scenario — `COPY . .` then `RUN rm secrets.txt` ; is the secret gone? No. The secret is in the earlier COPY layer; `rm` only adds a deletion in a later layer, recoverable via `docker history` /layer extraction. Correct: never copy it in — use BuildKit secrets (`--mount=type=secret`) or fetch at runtime, and multi-stage so it never reaches the final image.

17. Image tag vs digest; why is `:latest` dangerous? A tag (`:latest`) is a mutable pointer — two pulls can give different images, breaking reproducibility and rollbacks. A digest (`@sha256:...`) is immutable and content-addressed. **When to use:** Specific version tag → readable deploys. Digest pin → reproducible/immutable prod deploys and rollbacks. `:latest` → local dev only, never prod.

18. ENV vs ARG; which persists into the container? ARG = build-time only, not present in the running container. ENV = runtime variable baked into the image and present in the container. Neither should hold secrets. **When to use:** ARG → build-time params (versions, build flags). ENV → runtime config the app reads.

Runtime, Networking & Storage

19. Network drivers bridge / host / none / overlay? bridge = default isolated virtual network on one host with NAT. host = container shares the host's network stack (no isolation, no port mapping). none = no networking. overlay = multi-host network for Swarm/cluster. **When to use:** bridge (user-defined) → default multi-container on one host. host → max network perf / many dynamic ports, isolation not needed. none → no networking (batch/security). overlay → multi-host cluster.

20. Default bridge can't resolve by name, user-defined can — why? The default `bridge` has no built-in DNS service discovery (legacy `--link` only). A user-defined bridge runs Docker's embedded DNS, so containers resolve each other by name. **When to use:** User-defined bridge → always for multi-container. Default bridge → avoid.

21. `-p 8080:80` vs `--network host` ? `-p HOST:CONTAINER` → host port 8080 forwards to container port 80 (left = host). `--network host` removes that NAT layer — the container binds host ports directly. Host networking is faster (no NAT) but loses isolation and risks port conflicts. **When to use:** `-p` → normal, isolated, explicit mapping. host → high-throughput/low-latency or many/dynamic host ports.

22. Scenario — container can't resolve DNS, but the host can. Check the container's `/etc/resolv.conf`, the Docker daemon DNS settings, and whether you're on the default bridge (no service DNS). Common causes: Alpine/musl resolver quirks, corporate DNS not propagated to the daemon, custom network without DNS. Test with `docker run --dns 8.8.8.8`, `nslookup` inside, compare `resolv.conf`.

23. Why is container data ephemeral; where does the writable layer live? The writable layer is a thin copy-on-write layer destroyed when the container is removed; it lives in the storage driver (overlay2) on the host. Heavy writes there are slow (CoW) and lost on removal. **When to use:** Writable layer → ephemeral throwaway. Volume → any data that must persist or is write-heavy (DB, uploads).

24. cgroups vs namespaces — which does what? Namespaces = isolation (what a process can see: PID, network, mount, user, UTS, IPC). cgroups = resource limits (how much it can use: CPU, memory, I/O).

25. Limit memory/CPU; what happens on memory exceed? `--memory` / `-m` and `--cpus` (cgroups). Exceeding the memory limit triggers the kernel OOM killer → the container exits 137 (128+9). CPU limits throttle rather than kill.

Compose & Multi-Container

26. What does Docker Compose solve over `docker run` ? Declaratively defines a multi-container app (services, networks, volumes, env) in one YAML, brought up with one command. Replaces long, error-prone `docker run` chains. **When to use:** Compose → local/dev multi-container stacks, integration tests, simple single-host deploys. Move to Kubernetes → multi-host, scaling, self-healing prod.

27. `depends_on` vs actually ready; why it doesn't solve the race? `depends_on` controls start order, not readiness — it starts the DB container, not "DB accepting connections," so the app can start too early and crash. Fix: a DB healthcheck + `depends_on: { db: { condition: service_healthy } }` plus app-side retry.

28. How do Compose services discover each other? They share a user-defined network and resolve each other by service name via Docker's embedded DNS (e.g. the app connects to host `db`). No IPs or links needed.

29. `up` vs `up -d` vs `up --build` ? `up` = foreground (logs attached). `up -d` = detached/background. `up --build` = rebuild images before starting. **When to use:** up → foreground, watch logs (dev). up -d → background/normal run. up --build → after Dockerfile/source changes.

30. Pass config/secrets into Compose without hardcoding? `env_file:` / `environment:` from a `.env` (kept out of git), Docker secrets for sensitive data, or inject from the host/secret manager at runtime. Never commit secrets in the YAML.

Production, Security & Troubleshooting

31. Scenario — container crash-loops; full triage order. `docker ps -a` (exit code) → `docker logs` (app error) → `docker inspect` (OOMKilled, exit code, config) → check the entrypoint/CMD, env vars, missing mounts/config, dependencies. The exit code narrows it fast.

32. Scenario — `docker run` exits 0 immediately, container gone. The main process finished and exited — a container lives only as long as PID 1 runs. Likely the CMD is one-shot or the daemon backgrounds itself (forking, so PID 1 returns). Fix: run the process in the foreground (exec form, no `&`, `daemon off`).

33. Exit codes 137 / 125 / 126 / 127? 137 = 128+9, SIGKILL (often OOMKilled or `docker kill`). 125 = the Docker daemon/CLI itself failed (bad flag). 126 = command found but not executable (permissions). 127 = command not found (bad path/binary).

34. Why is running as root a problem; how to run non-root; what breaks? Container root maps toward host root — a breakout = host root. Use a non-root `USER`. What breaks: file ownership on mounted volumes, and binding ports < 1024 (needs root or `CAP_NET_BIND_SERVICE`) — so expose 8080 not 80, and `chown` files for the non-root UID.

35. PID 1 / zombie reaping; why `--init` or tini? PID 1 has special duties: reaping zombie children and forwarding signals. Most apps aren't written to do this, so zombies accumulate and signals drop. `--init` (or tini) inserts a tiny proper init as PID 1.

36. Why does shell-form CMD often not get SIGTERM on `docker stop`? Shell form runs `/bin/sh -c "your app"`, so the shell is PID 1 and your app is a child. `docker stop` sends SIGTERM to PID 1 (the shell), which doesn't forward it, so your app is SIGKILLed after the grace period. Use exec form so your app is PID 1.

37. Scenario — host out of disk, `/var/lib/docker` huge; safe cleanup; prune danger? Filling it: dangling/unused images, stopped containers, unused volumes, build cache. `docker system df` to see usage, then `docker container prune`, `docker image prune`, `docker builder prune`. Danger: `docker system prune -a --volumes` deletes all unused images and volumes — including data volumes you still need. Never run `--volumes` blindly on prod.

38. Docker healthcheck vs "process is up"; how the orchestrator uses it? A `HEALTHCHECK` tests application readiness (e.g. HTTP 200) → `healthy` / `unhealthy`, vs the process merely existing. Orchestrators use it to gate traffic, restart, or hold dependents until truly ready. **When to use:** Any service others depend on or that the orchestrator should gate traffic / restart on readiness.

39. Scan an image for vulnerabilities; CVE in a base image you don't control? Scan with Trivy/Grype/Docker Scout/ECR scanning in CI. For a base-image CVE: rebuild on a patched base tag, switch to a slimmer base (distroless/alpine) lacking the vulnerable package, pin a fixed digest once patched, and gate deploys on severity.

40. Scenario — same image works in staging, crashes in prod; both `:latest`. `:latest` is mutable — staging and prod may have pulled different images. Other culprits: env/config differences, secrets, resource limits (OOM in prod), network/dependency access, or platform/arch (`amd64` vs `arm64`). Fix: pin digests so both run the exact same image, then diff config/resources.

41. Container lifecycle created → running → exited; where do paused/restarting fit? `created` → `running` (PID 1 alive) → `exited` / `stopped` → `removed`. `paused` = processes frozen via the cgroup freezer (still in memory). `restarting` = the restart policy is relaunching it.

42. Why not store secrets in environment variables? Env vars leak via `docker inspect`, child processes, logs, crash dumps, and image layers if baked at build. Better: mount secrets as files (Docker/Swarm secrets, tmpfs), use a secrets manager (Vault, AWS Secrets Manager) at runtime, or BuildKit secrets for build-time. **When to use:** Mounted secret files / secrets manager → always for sensitive values. Env vars → non-sensitive config only. BuildKit secrets → secrets needed only at build time.

43. Scenario — CI build is slow, 12 min every time, no cache benefit. Order layers so rarely-changing steps (dependency install) come before source copy, use BuildKit cache mounts (`--mount=type=cache`) for package caches, and pull a registry cache with `--cache-from` / push `--cache-to`. Parallelize independent stages.

44. alpine vs distroless vs slim; when does Alpine bite? slim = stripped Debian, glibc, broad compatibility. alpine = tiny, musl libc instead of glibc. distroless = no shell/package manager, just runtime — most secure. Alpine bites with musl vs glibc incompatibilities (Python wheels recompiling, DNS quirks, locale/timing bugs). **When to use:** slim → broad compatibility, fewest surprises. alpine → smallest with a shell, watch musl issues. distroless → max security/minimal, no shell (harder to debug).

45. What is BuildKit; what does it add over the legacy builder? Parallel stage execution, better caching, cache mounts, build secrets (no leaking into layers), SSH forwarding, and SBOM/provenance. Default in modern Docker.

Conceptual / "Why" Closers

46. Where does Docker stop and orchestration (Kubernetes) begin? Docker builds/runs containers on one host and solves dependency packaging — but not scheduling, scaling, self-healing, or multi-host networking and state. Kubernetes orchestrates many containers across many hosts.

47. "Containers are lightweight VMs" — correct that. Wrong: a VM virtualizes hardware and runs a full guest OS kernel on a hypervisor. A container is just an isolated host process (namespaces + cgroups) sharing the host kernel — no guest OS. Lighter and faster, weaker isolation.

48. When would you NOT containerize something? Heavy stateful monoliths better served by managed services, GUI/desktop apps, workloads needing direct hardware/kernel-module access, or one-off single-server scripts where Docker adds operational overhead with no portability benefit.

49. Docker vs containerd vs runc vs the OCI spec? Docker = the daemon + CLI + build tooling (developer UX). containerd = the high-level runtime that manages container lifecycle/images. runc = the low-level OCI runtime that actually spawns the container process. OCI = the open spec they all implement. Kubernetes dropping "Docker as a runtime" just means it talks to containerd directly via CRI — your OCI images still run fine.

50. Two more sharp edges worth knowing. (a) Build context: `docker build .` ships the whole directory to the daemon — a missing `.dockerignore` can send gigabytes and secrets. (b) Time/UID/permission drift on bind mounts between host and container (especially non-root `USER`) causes "permission denied" that looks like an app bug but is really UID mismatch.

Notes compiled as a quick reference for Docker.